

IVISS

IVISS est une plateforme multi-tenant destinée aux équipes des forces de l'ordre et aux organismes de régulation pour effectuer des contrôles routiers, vérifier la conformité des véhicules et gérer les workflows d'application des sanctions.

Ce README est le point d'entrée pour les développeurs, opérateurs et reviewers. Il fournit une vue synthétique du système, du setup local, des frontières d'architecture, du modèle de sécurité, des conventions API/tests, et indique où trouver la documentation détaillée complète.

Table des matières

1. [Vue d'ensemble](#)
2. [Architecture et Décisions de Design](#)
3. [Stack Technique](#)
4. [Multi-tenancy et Modèle de Sécurité](#)
5. [Développement Local](#)
 - 5.1 [Prérequis](#)
 - 5.2 [Configuration \(`.env` \)](#)
 - 5.3 [Démarrage](#)
 - 5.4 [Commandes Utiles](#)
6. [Architecture Frontend](#)
7. [Architecture Backend \(Rust\)](#)
8. [Base de Données et Modèle de Données](#)
9. [Référence API et Conventions](#)
10. [Stratégie de Test](#)
11. [Qualité et Standards de Code](#)
12. [Debugging et Troubleshooting](#)

- 13. [Déploiement et CI/CD](#)
- 14. [Monitoring et Logging](#)
- 15. [Guide de Contribution](#)
- 16. [Appendices](#)

1. Vue d'ensemble

IVISS aide les agences à:

- Identifier les véhicules (saisie manuelle, workflows photo/OCR).
- Exécuter des opérations de contrôle terrain et conserver un historique traçable.
- Déclencher et suivre les actions de sanction.
- Opérer avec un contrôle d'accès par rôles dans un environnement multi-organisation.
- Utiliser une expérience web mobile-first avec capacités PWA.

Pour la vue système et le contexte fonctionnel, voir [docs/overview.md](#).

2. Architecture et Décisions de Design

Architecture de haut niveau:

- **Frontend:** React + TypeScript + Vite (SPA mobile-first, PWA activée)
- **Backend:** Rust + Axum + SQLx (API REST + OpenAPI)
- **Database:** PostgreSQL 15
- **Infrastructure:** Docker Compose (local), Terraform + Ansible + GitHub Actions (déploiement)

Décisions principales:

- Isolation des données multi-tenant par organisation.
- Contrat OpenAPI généré par le backend et consommé par le codegen frontend.
- Contrôle d'accès par rôles appliqué via middleware backend.
- Infrastructure-as-Code et pipelines CI/CD automatisés.

Pour les diagrammes et la justification détaillés, voir:

- [docs/architecture_spec.md](#)
- [docs/sequence_diagrams_&_flows.md](#)

3. Stack Technique

Frontend

- React, TypeScript, Vite
- Tailwind CSS + shadcn/ui (primitives Radix)
- TanStack Query, React Router
- Vitest + Testing Library

Backend

- Rust, Cargo, Axum, Tokio
- SQLx (PostgreSQL)
- Utoipa + Swagger UI (génération OpenAPI et documentation)

Plateforme

- Docker / Docker Compose
- GitHub Actions CI/CD
- AWS Lightsail (cible de déploiement)

4. Multi-tenancy et Modèle de Sécurité

Modèle central:

- Workflows et données scoppés par organisation.
- RBAC sur les rôles `admin` , `manager` (libellé métier: supervisor), `org_admin` et `agent` .
- Authentification JWT avec middleware d'accès orienté rôle.
- Workflows orientés audit pour les opérations sensibles.

Notes sécurité:

- Les providers externes (SMS/Email) sont configurés via variables d'environnement.
- Les secrets ne doivent jamais être commités.
- Les contrôles d'auth/session sont gérés côté serveur.

Pour le détail complet, voir:

- [docs/auth_tokens.md](#)
- [docs/developer/api.md](#)
- [docs/developer/coding-standards.md](#)

5. Développement Local

5.1 Prérequis

- Docker Engine 20.10+
- Docker Compose v2+
- Node.js 20+ (pour le développement frontend local et l'outillage)
- Toolchain Rust + Cargo (pour build/test backend en local)

5.2 Configuration (`.env`)

```
cp .env.example .env
```

Valeurs minimales pour démarrer le backend en local:

- `POSTGRES_PASSWORD`
- `EXTERNAL_POSTGRES_PASSWORD`
- `JWT_PRIVATE_KEY_PEM`
- `JWT_PUBLIC_KEY_PEM`
- `ACTIVATION_CODE_PEPPER`
- `SMS_PROVIDER`

Recommandé pour les workflows back-office de bout en bout:

- `EMAIL_PROVIDER` (`mock` , `resend` , `lettre` / `smtp`)
- Credentials du mode email sélectionné

Notes:

- La signification de ces variables d'environnement est correctement documentée dans le fichier `.env.example` .
- L'envoi réel d'email à l'org-admin (mot de passe temporaire) nécessite un provider réel (`resend` ou `lettre` / `smtp`) et des credentials valides.

Pour la matrice complète de setup, voir [docs/developer/getting-started.md](https://docs.developer/getting-started.md).

5.3 Démarrage

```
docker compose --profile dev up -d --build
```

URLs locales utiles:

- Frontend: <http://localhost:8080>
- Santé backend: <http://localhost:3000/api/v1/health>
- Swagger UI: <http://localhost:3000/docs>
- OpenAPI JSON: <http://localhost:3000/api-doc/openapi.json>
- Adminer: <http://localhost:8081>
- Santé metrics: <http://localhost:9091/health>

5.4 Commandes Utiles

```
# État des services
docker compose ps

# Logs
docker compose logs -f backend
docker compose logs -f frontend

# Stop stack
docker compose down

# Stop + suppression des volumes locaux (destructif)
docker compose down -v
```

Frontend hors Docker (optionnel):

```
cd frontend
npm install
npm run dev
```

6. Architecture Frontend

La structure frontend est organisée autour des routes, pages, composants réutilisables, hooks, et couches client OpenAPI générées.

Pour l'architecture frontend complète et le découpage développeur, voir:

- [docs/FE_ARCHITECTURE.md](#)
- [docs/developer/project-structure.md](#)

7. Architecture Backend (Rust)

Le backend est un service Axum avec des frontières modulaires pour handlers, services, queries, middleware, DTO et tests.

Points d'entrée clés:

- Runtime: `iviss-backend/src/main.rs`
- Routes: `iviss-backend/src/routes.rs`
- OpenAPI: `iviss-backend/src/api_doc.rs`

Pour la structure backend complète et les conventions, voir:

- [docs/developer/project-structure.md](#)
- [docs/developer/api.md](#)

8. Base de Données et Modèle de Données

IVISS utilise PostgreSQL avec migrations SQLx et mécanismes de seed.

Les domaines principaux incluent:

- Organisations et utilisateurs/rôles
- Registre véhicule et statuts
- Enregistrements de contrôle et actions
- Workflow de soumission en attente
- Logs d'audit

Pour la documentation complète schéma/DB, voir:

- [docs/schema.md](#)
- [docs/data.md](#)
- [docs/developer/database.md](#)

9. Référence API et Conventions

Le contrat API est généré depuis le code backend et exposé via OpenAPI.

Références:

- Swagger UI (local): <http://localhost:3000/docs>
- OpenAPI JSON (local): <http://localhost:3000/api-doc/openapi.json>
- Snapshot frontend: `frontend/openapi.json`

L'implémentation API et les conventions sont documentées dans:

- [docs/developer/api.md](#)

10. Stratégie de Test

Les tests sont structurés par sous-système:

- Backend: tests Rust, tests d'intégration, tests DB (Testcontainers)
- Frontend: tests unitaires/composants avec Vitest + Testing Library
- CI: validation par workflows pour build, lint, typage, tests, couverture et scans sécurité

Pour les commandes complètes et la correspondance CI, voir:

- [docs/developer/testing.md](#)

11. Qualité et Standards de Code

Les standards projet couvrent:

- Style de code et ownership des modules
- Discipline de changement API/DB
- Sécurité par défaut
- Conventional commits et hygiène PR

Voir:

- [docs/developer/coding-standards.md](#)

12. Debugging et Troubleshooting

Utiliser une approche de troubleshooting par couches:

- Santé des services et logs
- Validation env/config
- Vérifications auth/RBAC
- Problèmes de synchronisation OpenAPI/codegen

Voir:

- [docs/developer/debugging.md](#)

13. Déploiement et CI/CD

Modèle de déploiement production:

- Provisioning infrastructure: Terraform
- Configuration serveur et rollout: Ansible + Docker Compose
- Automatisation: GitHub Actions
- Cible: AWS Lightsail

Pour les procédures de déploiement et opérations complètes, voir:

- [docs/deployment_guide.md](#)
- [docs/release_guide.md](#)

14. Monitoring et Logging

L'observabilité opérationnelle inclut:

- Scraping Prometheus
- Dashboards Grafana
- Endpoints de santé services et logs conteneurs

Voir:

- [docs/monitoring.md](#)
- [monitoring/README.md](#)

15. Guide de Contribution

- Utiliser des branches de feature dédiées.
- Garder des PR ciblées et lisibles.
- Mettre à jour la doc dès qu'un comportement, une interface ou un workflow change.
- Exécuter les checks locaux pertinents avant d'ouvrir une PR.

Standards de contribution développeur:

- [docs/developer/coding-standards.md](#)

16. Appendices

A) Aperçu de la Structure du Projet

```
iviss/  
  .github/
```



```
workflows/
docs/
  architecture_spec.md
  developer/
    README.md
frontend/
  src/
    components/
    hooks/
    pages/
    router/
    services/
    openapi-rq/
  public/
  package.json
  vite.config.ts
  openapi.json
iviss-backend/
  src/
    handlers/
    services/
    queries/
    middleware/
    dto/
    tests/
    main.rs
    routes.rs
    api_doc.rs
  migrations/      # SQL migrations
  seeds/
  scripts/
  Cargo.toml
infra/
  terraform/
  ansible/
  scripts/
monitoring/
  prometheus/
  grafana/
scripts/
docker-compose.yml
.env.example
README.md
```

Pour la structure détaillée, voir docs/developer/project-structure.md.

B) Cartographie de la Documentation

- docs/overview.md
- docs/architecture_spec.md

- [docs/deployment_guide.md](#)
- [docs/developer/README.md](#)
- [docs/developer/getting-started.md](#)
- [docs/developer/api.md](#)
- [docs/developer/database.md](#)
- [docs/developer/testing.md](#)
- [docs/developer/coding-standards.md](#)
- [docs/developer/debugging.md](#)

C) Licence

Ce projet est propriétaire. Tous droits réservés.